

Software Lifecycle Management



Fachbereich Elektrotechnik
und Technische Informatik

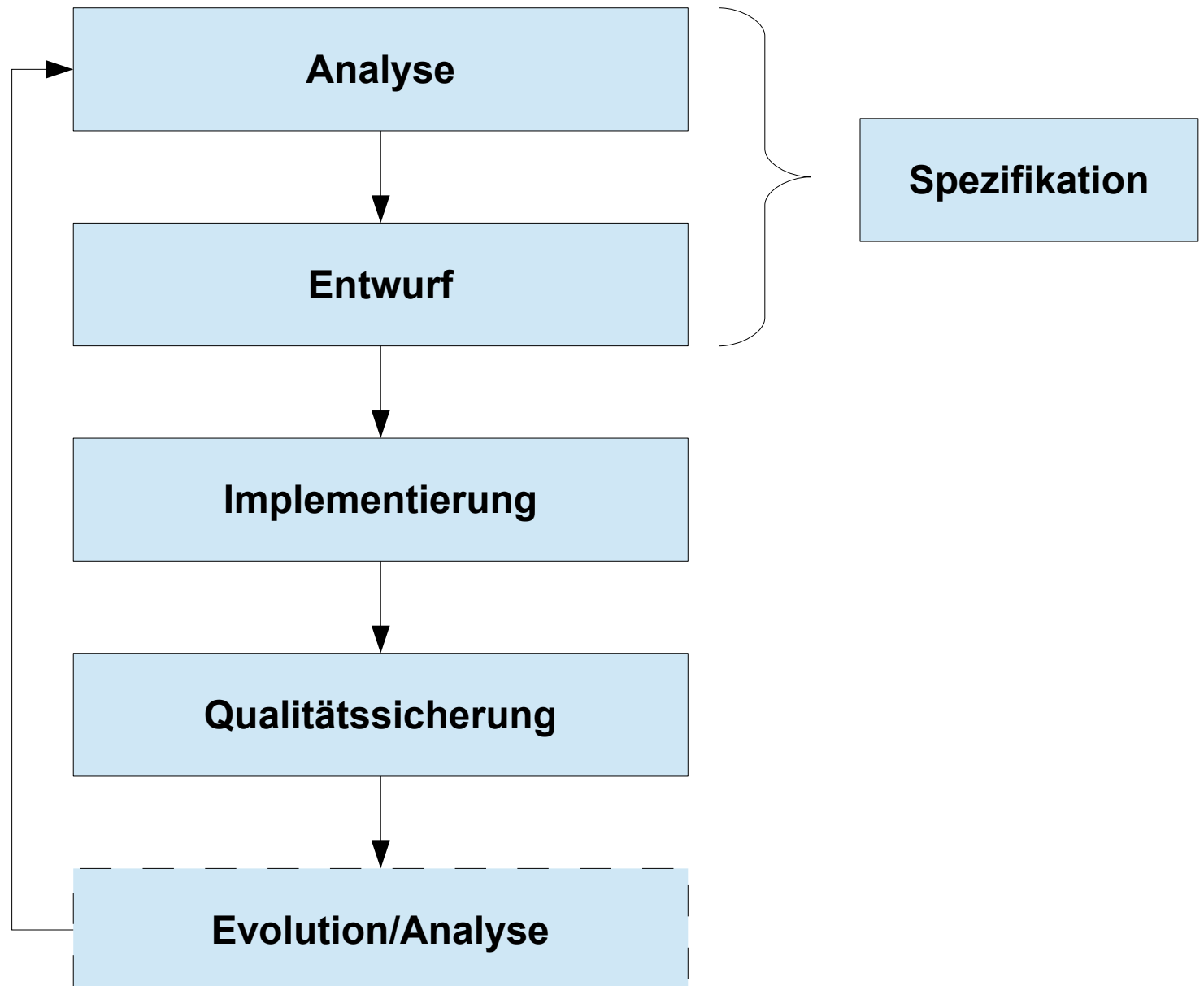
Dr. rer. nat. Nils Beckmann
nils.beckmann@th-owl.de

Regelwerke

Themen

- Rückblick: Kennengelernte Werkzeuge
- Weitere Werkzeuge
 - Regelwerke
 - Entindividualisierte Programmierung
 - Agiles Manifest
 - Verhalten des Softwareentwicklers
 - Lean Management

Erinnerung: Softwareprozessphasen



Kennengelernte Werkzeuge

- Application Lifecycle Management (ALM)
 - ALM-Management-Disziplinen
 - Software-Prozess
 - Software-Vorgehensmodell
 - Klassisch, Agil, Generisch
 - Phasen des Software Lifecycles:
Analyse, Entwurf, Implementierung, Qualitätssicherung

Kennengelernte Werkzeuge

- Requirements Management (REQ)
 - Softwareanforderungsspezifikation (SAS)
= Lastenheft + Pflichtenheft
 - Anforderung
 - Funktional, Nicht-Funktional
 - Produktqualitätsmatrix
 - Benutzer-A., System-A.
 - 9 Kriterien für Gute Anforderungen
 - Easy Approach to Requirements Syntax (EARS)
 - User Story (*user story*)
 - Anwendungsfall (*use case*)

Kennengelernte Werkzeuge

- Requirements Management (REQ)
 - Entwurf
 - 4 Kriterien für einen guten Entwurf
 - Modelle: Kontext, UML, MSC, Schichten, ...
- Issue and Defect Management (IDM)
 - Ticket
 - Statische/Dynamische Ticketeigenschaften
 - Anforderungs-Hierarchien
 - Epic, Feature, Backlog, Task
 - Fehlerbehandlung
 - Nachverfolgbarkeit ("Traceability")

Kennengelernte Werkzeuge

- Test and Quality Management (TQM)
 - Verifizierung & Validierung (V&V)
 - Statisch & Dynamisch
 - Testprozess
 - Testplan, Testbericht
 - Testfall, Modultest, Manuelles Testen
 - Testebenen, Testphasen
 - Regressionstest

Kennengelernte Werkzeuge

- Change and Configuration Management (CCM)
 - Wartung & Weiterentwicklung
 - Weiterentwicklungsprozesse
 - Portfolio-Analyse für Altsystem-Umgang
 - Änderungsantrag ("Request for Change", RfC)
 - Änderungsbericht ("Change Record", CR)
 - Konfigurations-Identifikationsdokument (KID)
- Variant Management (VM)
 - Varianten
 - Arten, Typen

Kennengelernte Werkzeuge

- Build and Release Management (BRM)
 - Build-Arten, Build-Server, "Nightly Build"
 - Release-Arten
 - Releasewechselplan
- Metrik (verschiedener Klassen)
 - Audit (Revision)
 - Report (Bericht)
- Resource Management (RM)
 - Ressource: Materiell, Immateriell
 - Netzplantechnik, Balkendiagramm, Planungsoptimierung

Weitere Werkzeuge

- Paarprogrammierung
- Testgetriebene Entwicklung
- Story-Card
- Entindividualisierte Programmierung
- Pull-Prinzip
- Lean Management
- Agiles Manifest
- Programmierrichtlinien
- ...

Weitere Werkzeuge

- ...
- Ethische Regeln
- Story Points
- Signalkarten (Kanban)
 - Aufgabekarteikarten
- Burndown-Diagramm
- Dokumentationsrichtlinien
- Formatvorlagen
- u.v.m.

Entindividualisierte Programmierung

- Idee: Nicht Einzelpersonen für Fehler verantwortlich sondern das ganze Team.

- Ziele:

Nach Weinberg, 1971

- Man achtet mehr aufeinander und auf die Arbeit des anderen.
- Produkt wird zur Gemeinschaftsleistung.
- Erfolge oder Rückschläge sind immer bezogen aufs ganze Team: Man feiert gemeinsam oder begibt sich gemeinsam auf Problemsuche.
- Das Individuum hat keine Verurteilung zu fürchten.

Das agile Manifest

- "Agile Manifesto"
 - Entwickelt 2001 im Rahmen der Entstehung der agilen Methoden
 - Weltweit tausende Unterzeichner aus der agilen Softwareentwicklung (als publizierte Selbstverpflichtung)
 - Entspricht Regelwerk bzw. Arbeitskodex (als Denkvorgabe)

Das agile Manifest

- "Wir erschließen bessere Wege, Software zu entwickeln, indem wir es selbst tun und anderen dabei helfen. Durch diese Tätigkeit haben wir diese Werte zu schätzen gelernt:
 - Individuen und Interaktionen stehen über Prozessen und Werkzeugen.
 - Funktionierende Software steht über einer umfassenden Dokumentation.
 - Zusammenarbeit mit dem Kunden steht über der Vertragsverhandlung.
 - Reagieren auf Veränderung steht über dem Befolgen eines Plans.
- Das heißt, obwohl wir die Werte auf der rechten Seite wichtig finden, schätzen wir die Werte auf der linken Seite höher ein."

Verhalten des Software-Engineers

- Beispiel: "Ethische Regeln und professionelles Verhalten des Software Engineering" (ACM/IEEE-CS)
- "[...] Software-Entwickler sollen sich verpflichten, Analyse Spezifikation, Entwurf, Entwicklung, Test und Wartung von Software zu einem nützlichen und geachteten Beruf zu machen. In Übereinstimmung mit ihren Verpflichtungen gegenüber der Gesundheit, Sicherheit und dem Wohlergehen der Öffentlichkeit sollen Software-Entwickler sich an die folgenden **Acht Prinzipien** halten:"
 - 1. Öffentlichkeit** – Software-Entwickler sollen in Übereinstimmung mit dem öffentlichen Interesse handeln.
 - 2. Kunde und Arbeitgeber** – Software-Entwickler sollen auf eine Weise handeln, die im Interesse ihrer Kunden und ihres Arbeitgebers ist und sich mit dem öffentlichen Interesse deckt.

...

Verhalten des Software-Engineers

- Fortsetzung: "Ethische Regeln und professionelles Verhalten des Software Engineering" (ACM/IEEE-CS)

...

- 3. Produkt** – Software-Entwickler sollen sicherstellen, dass ihre Produkte und damit zusammenhängende Modifikationen den höchstmöglichen professionellen Standards entsprechen.
- 4. Beurteilung** – Software-Entwickler sollen bei der Beurteilung eines Sachverhaltes Integrität und Unabhängigkeit wahren.
- 5. Management** – Für das Software Engineering verantwortliche Manager und Projektleiter sollen sich bei ihrer Tätigkeit ethischen Grundsätzen verpflichtet fühlen und in diesem Sinne handeln.
- 6. Beruf** – Software-Entwickler sollen die Integrität und den Ruf des Berufs in Übereinstimmung mit dem öffentlichen Interesse fördern.

...

Verhalten des Software-Engineers

- Fortsetzung: "Ethische Regeln und professionelles Verhalten des Software Engineering" (ACM/IEEE-CS)

...

7. Kollegen – Software-Entwickler sollen sich ihren Kollegen gegenüber fair und hilfsbereit verhalten.

8. Selbst – Software-Entwickler sollen sich einem lebenslangen Lernprozess in Bezug auf ihren Beruf unterwerfen und anderen eine ethische Ausübung des Berufs vorleben.

 Regelwerk für das Denken und Handeln

 Ziele: Gute Ergebnisse, Gutes Miteinander, Gute Darstellung.

Goldene Regel „Alles nun, was ihr wollt, daß euch die Leute tun sollen, das tut ihr ihnen auch.“
(Neues Testament, Matthäus-Evangelium, Kapitel 7, Vers 12)

Lean Management

„Schlankes Management“

- 1) Alle Tätigkeiten auf Kunden ausrichten
 - 2) Auf eigene Stärken konzentrieren
 - 3) Geschäftsprozesse optimieren
 - 4) Qualität ständig verbessern
 - 5) Interne Kundenorientierung als Unternehmensleitbild
 - 6) Eigenverantwortung, "Empowerment", Teamarbeit
 - 7) Dezentrale, kundenorientierte Strukturen
 - 8) Führen ist Service am Mitarbeiter
 - 9) Offene Informations- und Feedback-Prozesse
 - 10) Einstellungs- und Kulturwandel im Unternehmen
- Sammlung von sinnvollen, konstruktiven Maßnahmen zur Erzielung von besseren Ergebnissen.
 - Konkrete Umsetzung fehlt und muss individuell definiert werden.

Zusammenfassung

- Das Application Lifecycle Management (ALM) bietet eine Vielzahl von Werkzeugen und Methoden an
 - Diese unterstützen beim Verwalten und Steuern des "Software Lifecycles"
 - Dazu zählen auch verschiedene Regelwerke, u.A.
 - Die Entindividualisierte Programmierung stellt die Teamleistung in den Vordergrund
 - Das Agile Manifest hebt den agilen Prozess zur Entwicklung von Software hervor
 - Ethische Regeln fördern das gute Miteinander.
 - Das Lean Management möchte Entwicklung konstruktiv ausrichten für bessere Ergebnisse

Fragen?